

TUTORIAL PARA CRIAÇÃO DO SISTEMA DE LOGIN COM ADMINISTRADOR E USUÁRIOS

(Baseado no código final apresentado)

1. Objetivo

O aluno irá criar um sistema simples com menu, login, cadastro de usuários, área interna para usuário comum e área interna para administrador. O sistema também terá proteção contra erros através de try/except.

Esse programa ensinará na prática o uso de funções, laços, dicionários, verificações e menus interativos.

2. Estrutura geral do programa

Antes de começar a escrever o código, é importante entender a divisão:

3. Função `mostrar_menu_principal()`
4. Função `cadastrar_usuario(usuario)`
5. Função `login(usuario)`
6. Função `menu_interno_admin(usuario)`
7. Função `menu_interno_usuario(nome, usuario)`
8. Função principal `rodar_sistema()`
9. Chamada final para iniciar o sistema

Essas funções conversam entre si e formam o sistema completo.

3. Criando o arquivo no Google Colab

4. Abra o Colab.
5. Crie um novo notebook.
6. Na primeira célula você vai escrever todo o código.

7. Não divida em várias células para evitar erros de funções não encontradas.

4. Implementação passo a passo

A seguir, o aluno deve ir escrevendo cada parte exatamente na ordem.

4.1. Importando o módulo de tratamento de erros

Logo no início do seu código, escreva:

```
import traceback
```

Isso permitirá que o programa exiba erros completos caso algo falhe.

4.2. Criando o menu principal

Escreva uma função responsável por mostrar o menu inicial e receber a escolha do usuário:

```
def mostrar_menu_principal():  
    print("===== SISTEMA =====")  
    print("1 - Login")  
    print("2 - Cadastrar usuário")  
    print("3 - Sair")  
    return input("Escolha uma opção: ").strip()
```

Essa função devolve a opção selecionada pelo usuário.

4.3. Criando a função de cadastro de usuário

O cadastro fica armazenado dentro de um dicionário chamado usuarios.

Essa função deve:

- Perguntar o nome
- Verificar se está vazio
- Verificar se já existe
- Perguntar a senha

- Registrar

Escreva:

```
def cadastrar_usuario(usuarios):
    while True:
        nome = input("Digite o nome de usuário: ").strip()
        if nome == "":
            print("Nome não pode ficar vazio. Tente novamente.")
            continue
        if nome in usuarios:
            print("Usuário já existe. Escolha outro nome.")
            return
        break

    while True:
        senha = input("Digite a senha: ").strip()
        if senha == "":
            print("Senha não pode ficar vazia. Tente novamente.")
            continue
        break

    usuarios[nome] = {"senha": senha, "tipo": "usuario"}
    print("Usuário cadastrado com sucesso.")
```

4.4. Criando a função de login

Essa função deve verificar se o usuário existe e se a senha está correta.

```
def login(usuarios):
    nome = input("Usuário: ").strip()
    senha = input("Senha: ").strip()
    if nome in usuarios and usuarios[nome]["senha"] == senha:
        print("Login realizado com sucesso.")
        return nome
    else:
        print("Usuário ou senha incorretos.")
        return None
```

Se o login der certo, retornamos o nome do usuário.

Se não der, retornamos None.

4.5. Criando o menu interno do administrador

Esse menu só aparece para quem tem tipo "admin".

Funções do administrador:

1. Listar usuários
2. Excluir usuário
3. Logout

Escreva:

```
def menu_interno_admin(usuarios):
    while True:
        print("\n--- MENU DO ADMIN ---")
        print("1 - Listar usuários")
        print("2 - Excluir usuário")
        print("3 - Logout")
        opcao = input("Escolha: ").strip()

        if opcao == "1":
            print("\nUsuários cadastrados:")
            for u, d in usuarios.items():
                tipo = d.get('tipo') if isinstance(d, dict) else
str(d)
                print(f"- {u} ({tipo})")

        elif opcao == "2":
            alvo = input("Nome do usuário a excluir: ").strip()
            if alvo == "admin":
                print("Não é permitido excluir o administrador.")
            elif alvo in usuarios:
                del usuarios[alvo]
                print("Usuário removido com sucesso.")
            else:
                print("Usuário não encontrado.")

        elif opcao == "3":
            print("Logout do administrador.")
            break
```

```
else:
    print("Opção inválida. Tente novamente.")
```

4.6. Criando o menu interno do usuário comum

O usuário comum só pode ver seus próprios dados e fazer logout.

```
def menu_interno_usuario(nome, usuarios):
    while True:
        print("\n--- MENU DO USUÁRIO ---")
        print("1 - Meus dados")
        print("2 - Logout")
        opcao = input("Escolha: ").strip()

        if opcao == "1":
            if nome in usuarios and isinstance(usuarios[nome],
dict):
                print(f"\nUsuário: {nome}")
                print(f"Tipo: {usuarios[nome].get('tipo')}")
            else:
                print("Erro: dados do usuário não encontrados.")
        elif opcao == "2":
            print("Logout realizado.")
            break
        else:
            print("Opção inválida. Tente novamente.")
```

4.7. Criando a função principal do sistema

Essa função organiza todas as outras, controla o fluxo do programa, e trata erros gerais.

Escreva:

```
def rodar_sistema():
    try:
        usuarios = {
            "admin": {"senha": "123", "tipo": "admin"}
        }

        while True:
            opcao = mostrar_menu_principal()
```

```
        if opcao == "1":
            usuario_logado = login(usuario_logado)
            if usuario_logado:
                tipo = usuarios.get(usuario_logado,
{}}).get("tipo")
                if tipo == "admin":
                    menu_interno_admin(usuarios)
                else:
                    menu_interno_usuario(usuario_logado,
usuarios)

            elif opcao == "2":
                cadastrar_usuario(usuarios)

            elif opcao == "3":
                print("Encerrando o sistema.")
                break

        else:
            print("Opção inválida. Tente novamente.")

    except Exception:
        print("Ocorreu um erro inesperado. Veja o traceback
abaixo:")
        traceback.print_exc()
```

4.8. Executando o sistema

A última linha deve ser escrita para iniciar o programa:

```
rodar_sistema()
```

Sem essa linha o sistema não inicia.

5. O que o aluno deve testar ao finalizar

6. Tentar logar como admin

Usuário: admin

Senha: 123

7. Criar novos usuários
8. Logar como usuário comum
9. Listar usuários
10. Excluir usuários (menos o admin)
11. Testar entradas inválidas
 - opções erradas no menu
 - usuários vazios
 - senhas vazias
 - usuário repetido